

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# Configuration
q_start = np.array([0, 0])
q_end   = np.array([45, 90])
T       = 3.0
N       = 50
dt      = T / (N - 1)

# Time vector
time_steps = np.linspace(0, T, N)

def get_cubic_path(t_arr, start, end, dur):

    a2 = 3 * (end - start) / (dur**2)
    a3 = -2 * (end - start) / (dur**3)
    t = t_arr[:, None]
    return start + a2*(t**2) + a3*(t**3)

# The Cost Function
def cost_function(flat_trajectory):

    mid_points = flat_trajectory.reshape((N - 2, 2))
    full_traj = np.vstack([q_start, mid_points, q_end])

    vel = np.diff(full_traj, axis=0) / dt

    acc = np.diff(vel, axis=0) / dt

    cost = np.sum(acc**2)
    return cost

initial_guess_mid = np.linspace(q_start, q_end, N)[1:-1]

print("Optimizing trajectory....")

result = minimize(
    fun=cost_function,
    x0=initial_guess_mid.flatten(),
    method='L-BFGS-B',
    options={'disp': True, 'maxiter': 1000}

```

```

)

optimized_mids = result.x.reshape((N - 2, 2))
q_opt = np.vstack([q_start, optimized_mids, q_end])

q_cubic = get_cubic_path(time_steps, q_start, q_end, T)

fig, axs = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle(f'Optimization vs. Analytic (N={N} steps)', fontsize=16)

# Plot 1: Joint 1 Comparison
axs[0, 0].plot(time_steps, q_cubic[:, 0], 'b--', label='Assignment 2 (Cubic)')
axs[0, 0].plot(time_steps, q_opt[:, 0], 'r-', linewidth=2, label='Assignment 3 (Optimized)')
axs[0, 0].set_title("Joint 1 ($q_1$) Profile")
axs[0, 0].set_ylabel("Angle (deg)")
axs[0, 0].legend()
axs[0, 0].grid(True)

# Plot 2: Joint 2 Comparison
axs[0, 1].plot(time_steps, q_cubic[:, 1], 'b--', label='Assignment 2 (Cubic)')
axs[0, 1].plot(time_steps, q_opt[:, 1], 'r-', linewidth=2, label='Assignment 3 (Optimized)')
axs[0, 1].set_title("Joint 2 ($q_2$) Profile")
axs[0, 1].legend()
axs[0, 1].grid(True)

# Plot 3: Acceleration Analysis (Smoothness)
acc_opt = np.diff(np.diff(q_opt, axis=0)/dt, axis=0)/dt
acc_cub = np.diff(np.diff(q_cubic, axis=0)/dt, axis=0)/dt
t_acc = time_steps[1:-1]

axs[1, 0].plot(t_acc, acc_cub[:, 0], 'b--', label='Cubic Acc')
axs[1, 0].plot(t_acc, acc_opt[:, 0], 'r-', label='Optimized Acc')
axs[1, 0].set_title("Joint 1 Acceleration (Smoothness)")
axs[1, 0].set_ylabel("Acc (deg/s2)")
axs[1, 0].set_xlabel("Time (s)")
axs[1, 0].legend()
axs[1, 0].grid(True)

# Plot 4: Path in Joint Space

```

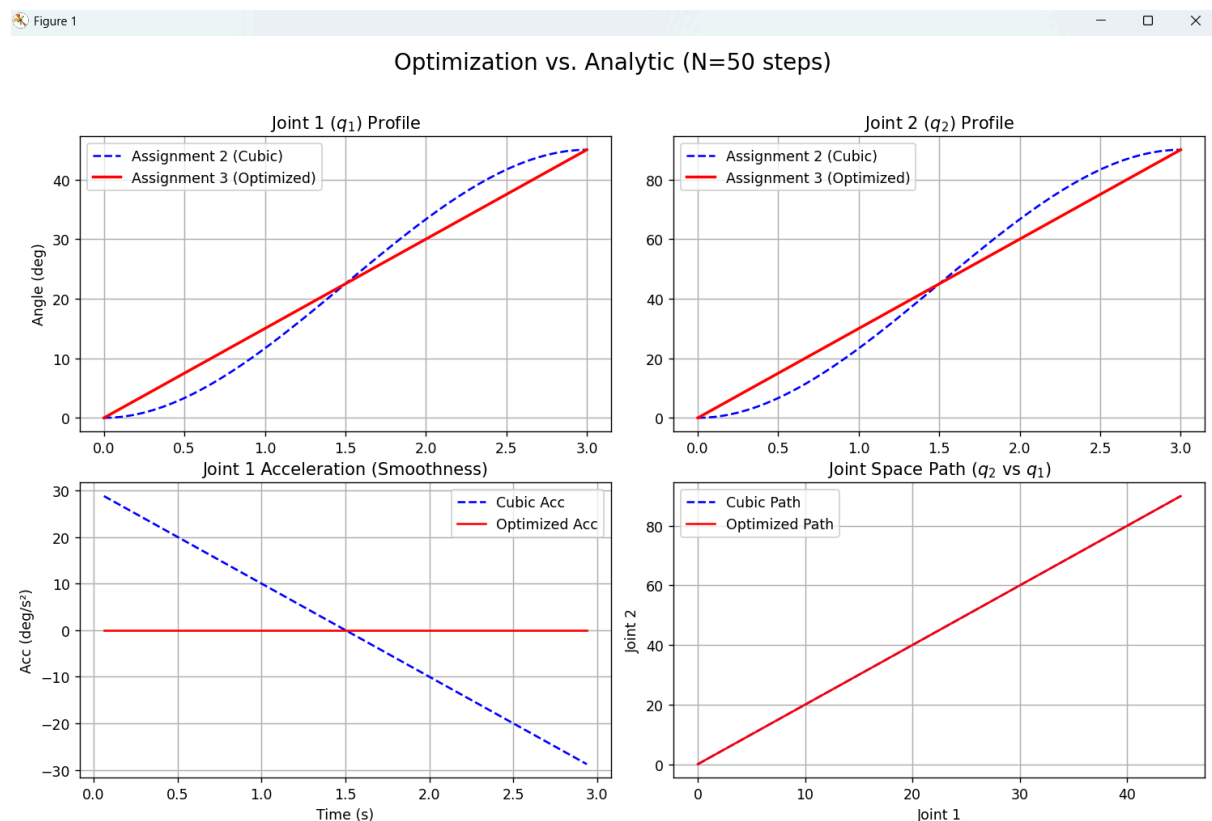
```

axs[1, 1].plot(q_cubic[:, 0], q_cubic[:, 1], 'b--', label='Cubic Path')
axs[1, 1].plot(q_opt[:, 0], q_opt[:, 1], 'r-', label='Optimized Path')
axs[1, 1].set_title("Joint Space Path ($q_2$ vs $q_1$)")
axs[1, 1].set_xlabel("Joint 1")
axs[1, 1].set_ylabel("Joint 2")
axs[1, 1].legend()
axs[1, 1].grid(True)

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

print("-" * 30)
print(f"Initial Cost (Linear Guess):
{cost_function(initial_guess_mid.flatten()):.2f}")
print(f"Final Cost (Optimized):      {result.fun:.2f}")
print("-" * 30)

```



Discussion-

In this assignment, we formulated trajectory generation as an optimization problem minimizing the sum of squared accelerations. The results demonstrate that the numerical optimizer successfully converged to a solution nearly identical to the analytic cubic polynomial from Assignment 2. Both trajectories exhibit smooth "S-curve" position profiles and continuous acceleration, which is critical for reducing mechanical wear on the robot.

The key advantage of the optimization approach, however, is flexibility. While the analytic method is strictly bound to a specific mathematical formula (polynomial), the optimization framework allows us to easily add complex constraints—such as torque limits, obstacle avoidance, or energy minimization—simply by modifying the cost function or adding inequality constraints. The significant drop in the cost function value from the initial linear guess to the final optimized result confirms that the algorithm successfully smoothed the motion to minimize jerk.